Αλγόριθμος LVQ2- LFM

Ο αλγόριθμος LVQ2 (Learning Vector Quantization 2) είναι ένας εποπτευόμενος αλγόριθμος μάθησης που χρησιμοποιείται για εργασίες ταξινόμησης. Τα βήματα του αλγορίθμου LVQ2 είναι τα εξής:

1. Αρχικοποιήσε τα πρωτότυπα διανύσματα, τα οποία αντιπροσωπεύουν τα κεντροειδή των κλάσεων στον χώρο χαρακτηριστικών.
2. Εισήγαγε ένα δείγμα εκπαίδευσης και υπολογίστε την Ευκλείδεια απόσταση μεταξύ του δείγματος και των πρωτοτύπων διανυσμάτων.
3. Αντιστοίχισε το δείγμα εκπαίδευσης στην τάξη που αντιπροσωπεύεται από το πλησιέστερο πρωτότυπο διάνυσμα.
4. Ενημέρωσε το πρωτότυπο διάνυσμα για την τάξη στην οποία καταχωρήθηκε το δείγμα εκπαίδευσης, μετακινώντας το πιο κοντά στο δείγμα εκπαίδευσης.
5. Επανάλαβε τα βήματα 2-4 για όλα τα δείγματα εκπαίδευσης στο σύνολο δεδομένων.
6. Επανάλαβε τα βήματα 2-5 για έναν καθορισμένο αριθμό επαναλήψεων ή έως ότου τα πρωτότυπα διανύσματα συγκλίνουν.

Είναι σημαντικό να σημειωθεί ότι ο αλγόριθμος LVQ2 είναι μια παραλλαγή του LVQ, που χρησιμοποιεί έναν όρο μείωσης βάρους, ο οποίος μειώνει τον ρυθμό εκμάθησης με την πάροδο του χρόνου, προκειμένου να αποφευχθεί η υπερβολική προσαρμογή.

Στο LVQ2, τα πρωτότυπα διανύσματα ενημερώνονται μόνο για την κλάση στην οποία ανήκει το δείγμα. Χρησιμοποιεί δύο διαφορετικά πρωτότυπα διανύσματα για κάθε τάξη, που ονομάζονται "code books", για να αναπαραστήσουν τα δεδομένα εκπαίδευσης. Ένα πρωτότυπο διάνυσμα χρησιμοποιείται για να αναπαραστήσει την πλειοψηφία των δεδομένων εκπαίδευσης, ενώ το άλλο πρωτότυπο διάνυσμα χρησιμοποιείται για να αναπαραστήσει τη μειοψηφία των δεδομένων εκπαίδευσης. Αυτό επιτρέπει στο LVQ2 να χειρίζεται καλύτερα μη ισορροπημένα σύνολα δεδομένων, όπου η μία κλάση υποεκπροσωπείται σε σύγκριση με την άλλη κλάση.

Το Adaptive LVQ2, ή ALVQ2, είναι μια παραλλαγή του LVQ2 που προσαρμόζει προσαρμοστικά τα πρωτότυπα διανύσματα κατά τη διάρκεια της προπόνησης για να αναπαραστήσουν καλύτερα τα δεδομένα εκπαίδευσης. Αυτό γίνεται με την ενημέρωση των πρωτοτύπων διανυσμάτων με βάση το σφάλμα μεταξύ της προβλεπόμενης τάξης και της πραγματικής κλάσης κάθε δείγματος εκπαίδευσης.
Δεν είναι δυνατόν να χρησιμοποιηθεί για τα δεδομένα που δόθηκαν λόγω του μικρού πλήθους των δοσμένων δεδομένων και της μικρής πολυπλοκότητας που αυτά διαθέτουν.

Συνοπτικά, οι LVQ2 και ALVQ2 είναι αλγόριθμοι επίβλεψης μάθησης που χρησιμοποιούνται για εργασίες ταξινόμησης και μπορούν να χειριστούν μη ισορροπημένα σύνολα δεδομένων χρησιμοποιώντας δύο πρωτότυπα διανύσματα ανά τάξη για να αναπαραστήσουν τα δεδομένα εκπαίδευσης.

Σημαντικό να αναφερθεί στην λειτουργία του αλγορίθμου LVQ2 η προσπάθεια της εξασφάλισης ενός χαμηλού ρυθμού εκμάθησης. Ένας χαμηλότερος ρυθμός εκμάθησης με

την πάροδο του χρόνου χρησιμοποιείται γενικά για την αποφυγή της υπερπροσαρμογής. Ο ρυθμός εκμάθησης ελέγχει το μέγεθος βήματος της κάθε ενημέρωσης που δέχονται τα χρησιμοποιούμενα πρότυπα διανύσματα κατά τη διάρκεια της εκπαίδευσης. Ένα υψηλό ποσοστό εκμάθησης σημαίνει ότι τα πρωτότυπα διανύσματα ενημερώνονται πιο επιθετικά, ενώ ένα χαμηλό ποσοστό εκμάθησης σημαίνει ότι οι ενημερώσεις είναι μικρότερες.

Όταν ο ρυθμός εκμάθησης είναι υψηλός, τα πρωτότυπα διανύσματα μπορεί να υπερβούν τη βέλτιστη θέση και αυτό μπορεί να οδηγήσει σε **υπερπροσαρμογή**. Για να αποφευχθεί η υπερβολική προσαρμογή, είναι σύνηθες να χρησιμοποιείται χαμηλότερος ρυθμός εκμάθησης καθώς προχωρά η εκπαίδευση, έτσι ώστε οι ενημερώσεις να γίνονται μικρότερες και τα πρωτότυπα διανύσματα να συγκλίνουν σε μια πιο σταθερή θέση.

Σημειώνεται ότι ο βέλτιστος ρυθμός εκμάθησης θα εξαρτηθεί από το συγκεκριμένο σύνολο δεδομένων και το πρόβλημα που μελετάται. Μπορεί να χρειαστεί κάποιος πειραματισμός για να βρεθεί ο καλύτερος ρυθμός εκμάθησης που εξισορροπεί την αντιστάθμιση μεταξύ ακρίβειας και υπερπροσαρμογής. Παρακάτω από τα παραγόμενα διαγράμματα ελέγχονται αλλάζοντας τις παραμέτρους εκμάθησης την ομαλότητα της εκπαίδευσης κατά περίπτωση, σύμφωνα με τις προυποθέσεις που μόλις τέθηκαν.

Για τη διαδικασία εκμάθησης δικτύου LVQ2 επιλέχθηκε στα dataset που δόθηκαν να χρησιμοποιηθεί η μέθοδος cross validation. Αποτελεί μια τεχνική που περιλαμβάνει τη διαίρεση του συνόλου δεδομένων σε ένα σύνολο εκπαίδευσης και ένα σύνολο επικύρωσης(validation set) και τη χρήση του συνόλου εκπαίδευσης για την εκπαίδευση του μοντέλου και του συνόλου validation για την αξιολόγηση της απόδοσης του μοντέλου. Συνήθως χρησιμοποιείται σε μικρού μεγέθους σετ, όπως αυτά της εκφώνησης για να μοιραστεί το δείγμα κατά τυχαίο τρόπο σε 3 μέρη, training set validation set and test set και η εκπαίδευση να γίνει στην περίπτωσή μας με 10 τέτοιους τυχαίους διαχωρισμούς των δεδομένων για την βέλτιστη και πλήρη εκπαίδευση στο σύνολο των δεδομένων του training set. (10-fold cross-validation)
Αυτό επιτρέπει να αξιολογηθεί η ικανότητα γενίκευσης του μοντέλου ή πόσο καλά μπορεί να αποδώσει σε νέα, unseen δεδομένα(test σετ).

Στις μεθόδους LVQ (Learning Vector Quantization), τα patterns, ή prototypes per category, χρησιμοποιούνται για την αναπαράσταση των δεδομένων εκπαίδευσης που χρησιμοποιούνται για την ταξινόμηση νέων, δεδομένων που δεν έχει χρησιμοποιήσει ο αλγόριθμος (test set). Τα μοτίβα δημιουργούνται συνήθως επιλέγοντας αντιπροσωπευτικά παραδείγματα κάθε τάξης από τα δεδομένα εκπαίδευσης. Αυτά τα μοτίβα χρησιμοποιούνται στη συνέχεια για την εκπαίδευση του αλγόριθμου LVQ, ο οποίος μαθαίνει τα χαρακτηριστικά κάθε τάξης με βάση αυτά τα μοτίβα.

Στη συνέχεια, χρησιμοποιεί αυτή τη γνώση για να ταξινομήσει νέα σημεία δεδομένων με βάση την τάξη με την οποία μοιάζουν περισσότερο. Ο αριθμός των μοτίβων (ή αλλιώς ο πίνακας W) που χρειάζονται θα εξαρτηθεί από την πολυπλοκότητα των δεδομένων και τον αριθμό των κλάσεων που προσπαθούμε να ταξινομήσουμε. Γενικά, όσο περισσότερα μοτίβα, τόσο καλύτερα ο αλγόριθμος LVQ θα μπορεί να μάθει τα χαρακτηριστικά κάθε τάξης και να ταξινομήσει τα νέα δεδομένα με ακρίβεια. Ωστόσο, εάν υπάρχουν πάρα πολλά μοτίβα, ο αλγόριθμος LVQ μπορεί να γίνει υπερβολικά πολύπλοκος και να μην γενικεύεται καλά σε νέα δεδομένα. Έτσι, συνοπτικά, τα prototypes per category χρησιμοποιούνται για την

εκπαίδευση του αλγόριθμου LVQ, ενώ η διασταυρούμενη επικύρωση χρησιμοποιείται για την αξιολόγηση της απόδοσης του εκπαιδευμένου αλγορίθμου LVQ σε νέα δεδομένα.

Ο αριθμός των μοτίβων που χρειάζονται στην ταξινόμηση LVQ θα εξαρτηθεί από την πολυπλοκότητα των δεδομένων σας και τον αριθμό των κλάσεων που προσπαθείτε να ταξινομήσετε. Γενικά, όσο περισσότερα μοτίβα έχετε, τόσο καλύτερα ο αλγόριθμος LVQ θα μπορεί να μάθει τα χαρακτηριστικά κάθε τάξης και να ταξινομήσει τα νέα δεδομένα με ακρίβεια. Ωστόσο, εάν έχετε πάρα πολλά μοτίβα, ο αλγόριθμος LVQ μπορεί να γίνει υπερβολικά πολύπλοκος και να μην γενικεύεται καλά σε νέα δεδομένα.

Δεν υπάρχει απόλυτος και γρήγορος κανόνας για τον καθορισμό του βέλτιστου αριθμού μοτίβων προς χρήση, καθώς θα εξαρτηθεί από τα συγκεκριμένα χαρακτηριστικά των δεδομένων κάθε σετ που μελετάται. Ακολουθούν μερικές γενικές οδηγίες που θα μπορούσαμε να ακολουθήσουμε:

- Εάν διαθέτουμε ένα μεγάλο σύνολο δεδομένων με πολλά χαρακτηριστικά (δηλαδή, πολλές διαστάσεις), μπορεί να χρειαστούν περισσότερα μοτίβα για να καταγράψουμε την πολυπλοκότητα των δεδομένων.
- Εάν διαθέτουμε ένα μικρό σύνολο δεδομένων με λίγες δυνατότητες, μπορεί να έχετε καλή απόδοση με λιγότερα μοτίβα.
- Εάν έχουμε μεγάλο αριθμό κλάσεων, μπορεί να χρειαστούν περισσότερα μοτίβα για να αναπαραστήσουμε με ακρίβεια τα χαρακτηριστικά κάθε τάξης.
- Εάν έχουμε μικρό αριθμό τάξεων, μπορεί να έχουμε καλή απόδοση με λιγότερα μοτίβα.

Τελικά, ο καλύτερος τρόπος για να προσδιοριστεί ο βέλτιστος αριθμός μοτίβων που θα χρησιμοποιηθούν είναι να δοκιμάσουμε μια σειρά τιμών και να επιλέξουμε ποια έχει την καλύτερη απόδοση στο σύνολο δεδομένων μας. Χρησιμοποιήσαμε τη τεχνική της διασταυρούμενης επικύρωσης για να αξιολογήσουμε την απόδοση του αλγόριθμου LVQ για διαφορετικούς αριθμούς patterns και να επιλέξουμε αυτό που έχει την καλύτερη απόδοση.

Πριν παρουσιαστούν τα αποτελέσματα τέτοιων πειραμάτων, θα αναφερθούν τα αλγοριθμικά βήματα που εφαρμόστηκαν για τη δημιουργία του αλγορίθμου στο περιβάλλον matlab.

- Προστίθεται το αρχείο του data set που θα χρησιμοποιηθεί, είτε το iris.txt ή το wine.txt
- Το περιεχόμενο του αρχείου φορτώνεται στον πίνακα A.
- Αρχικοποιούνται τα μέγεθη των πινάκων σύμφωνα με τα δεδομένα που δόθηκαν. πχ. number of inputs, number of outputs number of patterns. Εδώ θα δηλωθούν ο αριθμος ppc οι εποχές εκπαίδευσης ο ρυθμός εκπαίδευσης ka και η αρχικοποίηση της συνάρτησης κόστους, a0. Επιπλέον, δηλώνεται το ποσοστό των συνολικών δεδομένων που κάθε φορά θα εκπαιδεύεται, που θα χρησιμοποιηθεί για validation και τέλος για set. Επιλέγεται το 70% να χρησιμοποιηθεί για training, 20% validation 10% για test set. Επειδή τα δεδομένα είναι στοιχισμένα, πρώτο βήμα επεξεργασίας είναι να τα μπερδέψουμε με τυχαία σειρά.
- Για κάθε ενα εκ των 10 fold cross-validation, δημιουργούμε πίνακες με τα indexes, θέσεις που βρίσκονται τα δεδομένα. Αν αναλογιστούμε ότι χωρίστηκαν σε 3 μέρη τα δεδομένα σε κάθε επανάληψη, τα πρώτα 70% ανήκουν στην εκπαίδευση, τα επόμενα 20% στο validation και τα υπόλοιπα 10% στο τεστ σετ. Βρέθηκε ένας αποδοτικός τρόπος να μοιραστούν τα δεδομένα για κάθε επανάληψη χρησιμοποιώντας την συν/ση mod, δλδ το ακέραιο υπόλοιπο της διαίρεσης κάθε φορά με τον συνολικό αριθμό δεδομένων που

θέλουμε να γεμίσει σε κάθε είδος σετ. Με τους πίνακες θέσεων για τα 3 σύνολα εύκολα πλέον δημιουργούνται οι πίνακες Ptr, Pval, Ptest των δεδομένων, καθώς τυχαία σύνολα χωρίστηκαν σε κάθε fold. Δεν ξεχνάμε να κρατήσουμε πίνακες με τα labels κάθε δεδομένου των 3ων συνόλων- για validation του LVQ2 που θα προγραμματιστεί.

- Αρχικοποίηση του πίνακα W με τα patterns per category που επιλέχθηκε να χρησιμοποιηθούν. Στον πίνακα L υπάρχουν τα labels για κάθε ένα pattern του πίνακα W. Τρέχει μία επανάληψη εντός στο σύνολο εκπαίδευσης που δέχεται δεδομένα από το σετ όποτε βρει σε μία επανάληψη ένα πρότυπο ίδιου Label με αυτό που χρειάζεται, έως ότου να αρχικοποιηθεί με κάποιες θέσεις δεδομένων όλα τα patterns στον πίνακα W. Προγραμματίστηκε έτσι ώστε να τρέχει κάθε φορά από εκεί που σταμάτησε και αρχικοποιήθηκε ένα πρότυπο και όχι να ξεκινάει σε κάθε επανάληψη από το στοιχείο 1 του πίνακα εκπαίδευσης. στόχος είναι να εξασφαλιστεί η τυχαιότητα επιλογής των προτύπων που χρησιμοποιήθηκαν και να προσπαθήσουμε να κάνουμε τον αλγόριθμο να μην επιλέγει τα ίδια πρότυπα κάθε φορά που τρέχει ο αλγόριθμος. Διαφορετικά ο πίνακας W τελικά θα είχε επιλεγμένα τα ίδια πρότυπα διπλά και τριπλά.
- Είμαστε πλέον έτοιμοι να ξεκινήσουμε την εκπαίδευση. Η εκπαίδευση θα γίνει για όσες εποχές δηλώσουμε ότι θα τρέξει ο αλγόριθμος εκπαιδευσης, 2 5 ή 10 φορές.

for ep=1:epochs

- Υπολογίζεται ο χρόνος εκπαίδευσης κάθε επανάληψης σύμφωνα με τον τύπο t=(ep-1)*trpats+it-1, η τιμή για την συνάρτηση a(t) και το ki που αποτελεί τον δείκτη ανάλογα με τη θέση του δεδομένου που εκπαιδεύεται τη στιγμή t.

for it=1:trpats

- Δημιουργείται ο πίνακας eukldist που μετατρέπει σε πίνακα αποστάσεων το κάθε σημείο από τον πίνακα δεδομένων στο σύνολο εκπαίδευσης από τον πίνακα W των πατερνσ που χρησιμοποιήθηκαν. Για να πραγματοποιηθεί η εκπαίδευση πρέπει ταυτόχρονα να ικανοποιούνται οι 3 περιορισμοί:Για ένα συγκεκριμένο σημείο εκπαίδευσης x(t), πρέπει να πληρούνται 3 προϋποθέσεις για να συμβεί εκμάθηση LVQ:
  1) Το πλησιέστερο πρωτότυπο στο x(t) πρέπει να είναι λάθος κλάσης wi
  if L(idx1) ~= tr_lab(ki) %Wlab(idx1) wrong class
  2) Το επόμενο πλησιέστερο πρωτότυπο στο x(t) πρέπει να είναι της σωστής κλάσης wj
  if L(idx2) == tr_lab(ki) %Wlab(idx2) correct class
  3) Το x(t) πρέπει να πέφτει μέσα σε ένα μικρό συμμετρικό παράθυρο γύρω από το μέσο του wi και του wj

if min([mindist1/mindist2,mindist2/mindist1])>1-s & max([mindist1/mindist2,mindist2/mindist1])<1+s

- Τότε πραγματοποιείται η εκπαίδευση για τα 2 κοντινότερα πατερνς του W.

W(:,idx1)=W(:,idx1)-at*(Ptr(:,ki)-W(:,idx1)); %vector for the incorrect category

W(:,idx2)=W(:,idx2)+at*(Ptr(:,ki)-W(:,idx2)); %vector for the correct category

end end end

Έπειτα βγάζουμε τα αποτελέσματα εκπαίδευσης για τα 3 σετ για κάθε μία εποχή εκπαίδευσης. Συγκεκριμένα ο πίνακας cv training cv validation, cv test Αποθηκεύει ανά εποχή για κάθε ένα εκ των 10 cross validation τα αποτελέσματα ταξινόμησης.

end

end

Διαγράμματα αποτελεσμάτων.

Ο LVQ (Learning Vector Quantization) και ο LFM (Learning From Memory) είναι και οι δύο αλγόριθμοι επίβλεψης μάθησης που χρησιμοποιούνται για εργασίες ταξινόμησης. Ωστόσο, έχουν κάποιες βασικές διαφορές ως προς τη δομή τους και τον τρόπο λειτουργίας τους.

Το LVQ είναι ένας τύπος νευρωνικού δικτύου που εκπαιδεύεται στην ταξινόμηση σημείων δεδομένων μαθαίνοντας τα πρωτότυπα ή τα «διανύσματα βιβλίου κωδικών» που αντιπροσωπεύουν κάθε κλάση. Το LVQ2 είναι μια βελτίωση σε σχέση με τον αρχικό αλγόριθμο LVQ, ο οποίος χρησιμοποιεί έναν όρο μείωσης βάρους για να αποτρέψει την υπερβολική προσαρμογή και να βελτιώσει τις δυνατότητες γενίκευσης του μοντέλου.

Από την άλλη πλευρά, το LFM (Learning from Memory) είναι ένας τύπος αλγόριθμου μάθησης που βασίζεται στη μνήμη, ο οποίος χρησιμοποιεί μια προσέγγιση πλησιέστερου γείτονα για την ταξινόμηση των σημείων δεδομένων. Αποθηκεύει ένα σύνολο labeled-παραδειγμάτων στη μνήμη και για κάθε νέα είσοδο, βρίσκει τους k-πλησιέστερους γείτονες στη μνήμη και εκχωρεί την είσοδο στην κλάση που έχει τα πιο αντιπροσωπευτικά παραδείγματα μεταξύ των γειτόνων της.

Συνοπτικά, η κύρια διαφορά μεταξύ LVQ2 και LFM είναι ότι η LVQ2 είναι μια μέθοδος που βασίζεται σε prototype-based μέθοδο και η LFM είναι μια μέθοδος που βασίζεται στη μνήμη. Το LVQ2 χρησιμοποιεί ένα νευρωνικό δίκτυο για να εκπαιδεύσει πρωτότυπα που αντιπροσωπεύουν τις κλάσεις, ενώ το LFM χρησιμοποιεί μια προσέγγιση πλησιέστερου γείτονα για να βρει την κλάση μιας νέας εισόδου με βάση την ομοιότητά της με τα παραδείγματα που είναι αποθηκευμένα στη μνήμη. Στα αλγοριθμικά βήματα αλλάζει μονάχα ο τρόπος και οι προϋποθέσεις εκπαίδευσης.

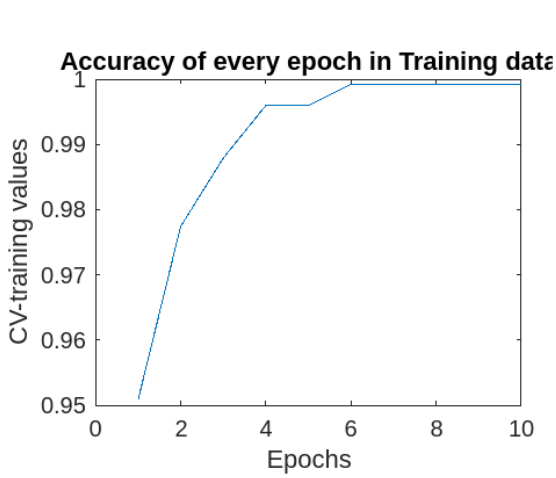
Παρακάτω δημιουργήθηκε πίνακας των αποτελεσμάτων των αλγορίθμων που προγραμματίστηκαν για τον LVQ2, χρησιμοποιώντας το wine set. **Ο τελικός πίνακας βαρών** μπορεί να δημιουργηθεί είτε με ένα mean W του πίνακα για όσες φορές εκπαιδεύτηκε, ή διατηρώντας στη μνήμη μόνο τον W πίνακα εκπαίδευσης με τα καλύτερα αποτελέσματα .

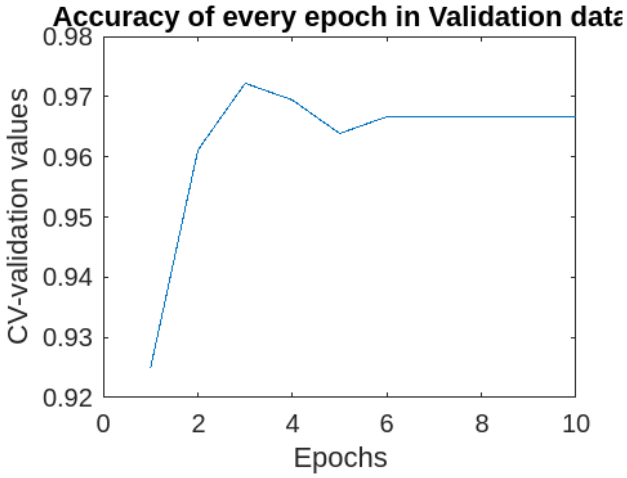
ppc=1	x	x	x	x	x	x												
ppc=2							x	x	x	x	x	x						
ppc=5													x	x	x	x	x	x
epoch=5	x			x			x			x			x			x		
epoch=10		x			x			x			x			x			x	
epoch=15			x			x			x			x			x			x
ka=0.1	x	x	x				x	x	x				x	x	x			

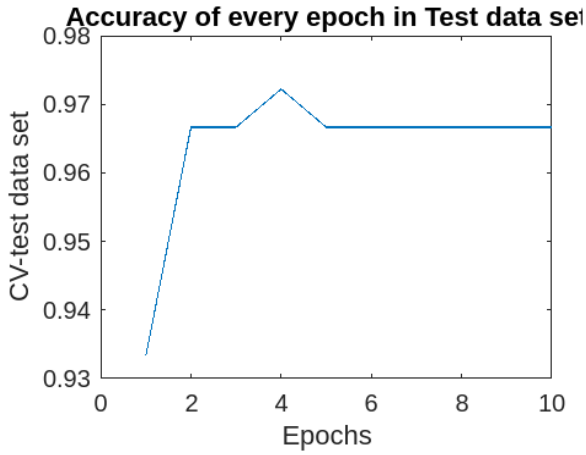
ka=0.01				x	x	x				x	x	x				x	x	x
training set	0.9773	0.98048	0.98486	9.8500	0.9946	0.99053	0.95006	0.98017	0.97006	9.729	9.763	9.731	0.97854	0.96554	0.96679	0.97992	0.98072	0.98003
validation set	0.95	0.95000	0.92491	9.594	0.9653	0.96057	0.91778	0.94222	0.92202	9.361	9.336	9.320	0.93028	0.91011	0.91167	0.94067	0.94036	0.94017
test set	0.9556	0.93328	0.94373	9.433	0.9656	0.95222	0.91000	0.93328	0.93326	9.133	9.433	9.333	0.96000	0.93389	0.87441	0.92333	0.95050	0.94041
time	3.007686	5.218585	7.708217	2.879587	5.090962	7.948639	2.779769	5.569428	8.305062	2.811.362	5.820.707	8.432.897	3.346173	6.647467	9.988761	3.375210	6.628893	10.028858

Στο wine.txt, for ppc=1 epochs=10 and ka=0.01 έχουμε τα καλύτερα αποτελέσματα χρονικά και ποιοτικά.

Τα γραφήματα είναι:

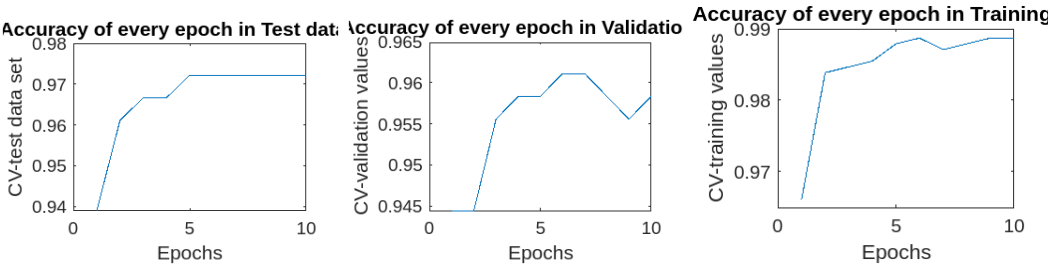






wine set	results after training with lfm				LFM													
ppc=1	x	x	x	x	x	x												
ppc=2							x	x	x	x	x	x						
ppc=5													x	x	x	x	x	x
epoch=5	x			x			x			x			x			x		
epoch=10		x			x			x			x			x			x	
epoch=15			x			x			x			x			x			x
ka=0.1	x	x	x				x	x	x				x	x	x			
ka=0.01				x	x	x				x	x	x				x	x	x
training set	0.9731	0.9725	0.9712	0.9592	0.9652	0.9661	0.9826	0.9847	0.9784	9.819	9.831	9.846	0.9827	0.9922	0.9894	0.9911	0.9918	0.9932
validation set	0.9611	0.9592	0.9561	0.9594	0.9483	0.9559	0.9628	0.9731	0.9441	9.561	9.719	9.626	0.9650	0.9603	0.9707	0.9556	0.9469	0.9563
test set	0.9611	0.9611	0.9556	0.9544	0.9550	0.9644	0.9511	0.9711	0.9611	9.533	9.811	9.711	0.9422	0.9644	0.9519	0.9711	0.9633	0.9570
time	2.626112	5.353647	7.807288	2.612065	5.252658	7.934789	2.821533	5.616848	8.557700	2.840.905	5.655.746	8.487.323	3.414921	6.753540	10.112734	3.414809	6.857552	10.170192

Στο wine.txt, ο LFM for ppc=2 epochs=10 and ka=0.01 έχουμε τα καλύτερα αποτελέσματα χρονικά και ποιοτικά.



Η απόδοση των LVQ2 και LFM εξαρτάται από το συγκεκριμένο πρόβλημα και το σύνολο δεδομένων που χρησιμοποιείται. Και οι δύο αλγόριθμοι έχουν χρησιμοποιηθεί σε ποικίλες εφαρμογές και έχουν δείξει καλά αποτελέσματα σε ορισμένες περιπτώσεις.

Ο LVQ2 είναι ένας αλγόριθμος που βασίζεται σε νευρωνικά δίκτυα, μπορεί να χειριστεί μη γραμμικά όρια απόφασης και δεδομένα υψηλών διαστάσεων και είναι καλός στο χειρισμό πολύπλοκων μοτίβων και θορύβου. Ωστόσο, απαιτεί μεγάλη ποσότητα επισημασμένων δεδομένων για την εκπαίδευση και είναι ευαίσθητο στην επιλογή υπερπαραμέτρων.

Από την άλλη πλευρά, το LFM είναι ένας αλγόριθμος που βασίζεται στη μνήμη, είναι καλός στο χειρισμό μικρών συνόλων δεδομένων, είναι απλός στην εφαρμογή του και είναι λιγότερο ευαίσθητος στην επιλογή υπερπαραμέτρων. Ωστόσο, το LFM δεν μπορεί να χειριστεί μη γραμμικά είδη απόφασης και δεδομένα υψηλών διαστάσεων όπως και το LVQ2.

Γενικά, το LVQ2 είναι πιο ισχυρό και ευέλικτο, αλλά απαιτεί περισσότερους υπολογιστικούς πόρους και περισσότερα δεδομένα με ετικέτα. Ο LFM είναι ένας πιο απλός αλγόριθμος, αλλά έχει περισσότερους περιορισμούς. Ανάλογα με το συγκεκριμένο πρόβλημα και το σύνολο δεδομένων, οποιοσδήποτε αλγόριθμος μπορεί να έχει καλύτερη απόδοση.

Είναι σημαντικό να σημειωθεί ότι και οι δύο αλγόριθμοι χρησιμοποιούνται σε συγκεκριμένους τομείς και δεν είναι δυνατό να πούμε ότι ο ένας είναι καλύτερος από τον άλλο γενικά, θα εξαρτηθεί από το πρόβλημα, το σύνολο δεδομένων και τον στόχο της ανάλυσης.

IRIS set	**results after training with LVQ2**				**LVQ2**													
ppc=1	x	x	x	x	x	x												
ppc=2							x	x	x	x	x	x						
ppc=5													x	x	x	x	x	x
epoch=5	x			x			x			x			x			x		
epoch=10		x			x			x			x			x			x	
epoch=15			x			x			x			x			x			x
ka=0.1	x	x	x				x	x	x				x	x	x			
ka=0.01				x	x	x				x	x	x				x	x	x
training set	0.8048	0.7969	0.8009	0.80084	0.8129	0.8168	0.79798	0.80142	0.78153	8.135	8.167	8.090	0.7979	0.8095	0.8135	0.8134	0.7964	0.8111
validation set	0.9555	0.9610	0.9615	0.9600	0.9500	0.9566	0.95529	0.93997	0.93335	9.523	9.552	9.234	0.9484	0.9413	0.9404	0.9613	0.9358	0.9327
test set	0.9375	0.95122	0.9467	0.9537	0.9613	0.9492	0.94225	0.94437	0.93113	9.400	9.462	9.279	0.9500	0.9494	0.9475	0.9563	0.9131	0.9071

time	1.804231	3.6643115	5.35995	1.811967	3.5809948	5.310765	1.881870	3.739415	5.579929	1.865496	3.798941	5.591219	2.255848	4.380746	6.584415	2.240135	4.362915	6.554503

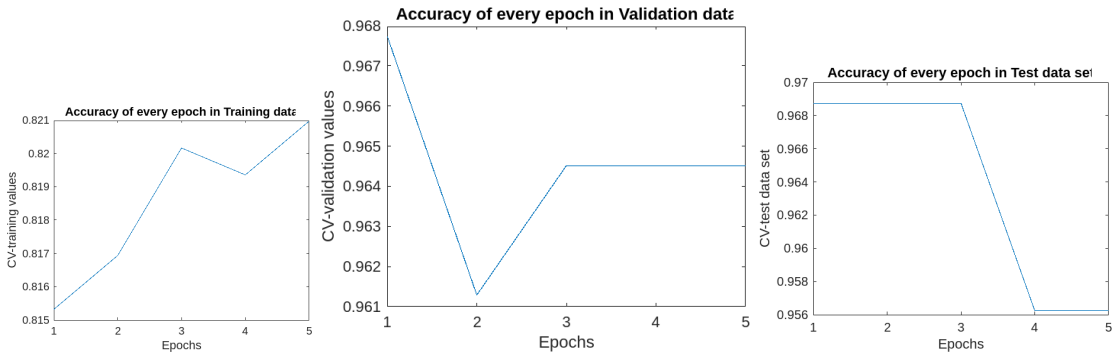
Στο iris.txt, for ppc=1 epochs=10 and ka=0.01 έχουμε τα καλύτερα αποτελέσματα χρονικά και ποιοτικά.

Γενικώς παρατηρείται ικανοποιητική ταξινόμηση των δεδομένων που συνήθως ξεπερνάει το 93%. Παρατηρείται ότι μερικές φορές μπορεί να γίνει overfitting των δεδομένων. Συγκεκριμένα επειδή τα 2 σετ δεν έχουν τεράστιο όγκο δεδομένων χωρίς μεγάλη πολυπλοκότητα των υπαρχόντων χαρακτηριστικών σε αυτά,είναι εύκολη η εκπαίδευσή του. Αν όμως αυξηθεί υπερβολικά ο αριθμός των εποχών μπορεί τελικά να έχουμε overfitting στα δεδομένα. Η υπερπροσαρμογή μπορεί να προκύψει εάν το μοντέλο είναι πολύ περίπλοκο για τον όγκο των δεδομένων και αρχίσει να απομνημονεύει τα δεδομένα εκπαίδευσης, αντί να γενικεύει σε “μη ορατά” δεδομένα. Για να αποφευχθεί η υπερβολική προσαρμογή, είναι σημαντικό να χρησιμοποιούνται τεχνικές όπως η τακτοποίηση και η πρόωρη διακοπή, και να παρακολουθείτε την απόδοση του μοντέλου σε ένα δεσμευμένο σετ επικύρωσης κατά τη διάρκεια της εκπαιδευτικής διαδικασίας. Επιπλέον, η χρήση k-fold cross-validation με k>10, μπορεί να βοηθήσει στη μείωση της πιθανότητας υπερβολικής προσαρμογής.

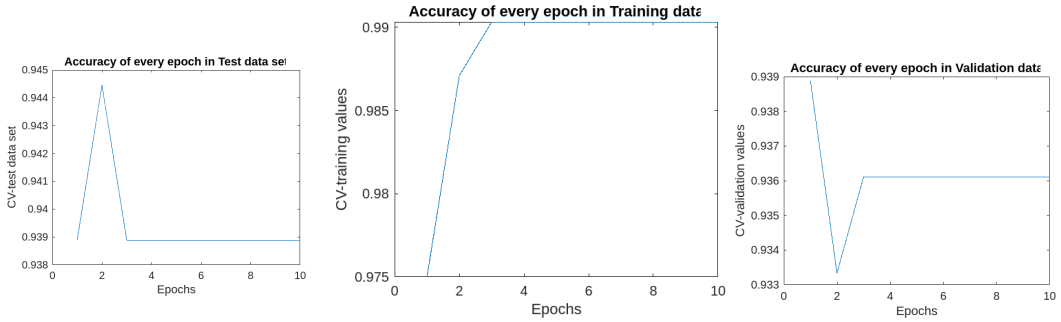
Η εργασία έχει ήδη τελειώσει, αλλά μιας και αρχικά για την εργασία θεώρησα ότι έπρεπε να παραχθούν, εδώ και κάτω παραθέτω γενικώς τα παραχθέντα μου αποτελέσματα και γραφήματα για την οπτικοποίηση και μελέτη της ταξινόμησης που λαμβάνει χώρα.
Σας ευχαριστώ πολύ

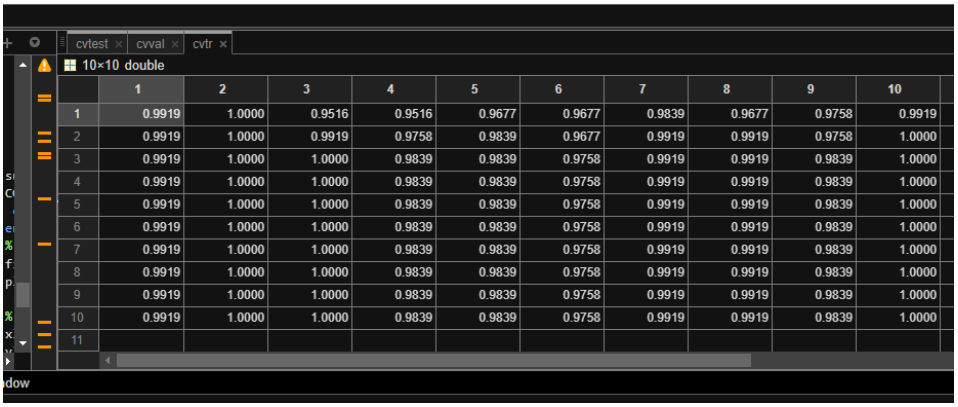
```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=5;
ka=0.01;
a0=0.5;


Elapsed time is 2.264820 seconds.
Number of epochs: 5, Number of patterns per category: 1, Rate of Learning:
0.010000
```



```
'WINE.TXT'
s=0.6;
ppc=5;
epochs=10
ka=0.01;
a0=0.5;
```





cvtest | cvval | cvtr

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.9919	1.0000	0.9516	0.9516	0.9677	0.9677	0.9839	0.9677	0.9758	0.9919
2	0.9919	1.0000	0.9919	0.9758	0.9839	0.9677	0.9919	0.9919	0.9758	1.0000
3	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
4	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
5	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
6	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
7	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
8	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
9	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
10	0.9919	1.0000	1.0000	0.9839	0.9839	0.9758	0.9919	0.9919	0.9839	1.0000
11										

LVQ2.m | cvtest | cvval

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.9167	0.8889	0.9722	0.9167	0.9444	1.0000	0.8889	0.9444	0.9444	0.9722
2	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
3	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
4	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
5	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
6	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
7	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
8	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
9	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
10	0.9167	0.8889	0.9722	0.9444	0.9722	1.0000	0.8889	0.8889	0.9444	0.9444
11										

Command Window

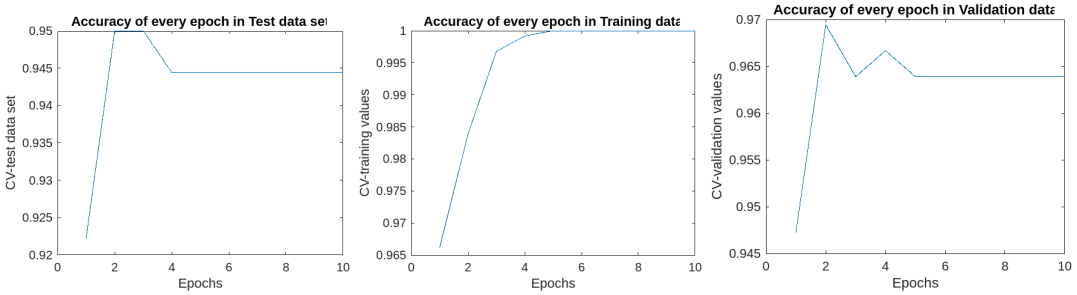
>>

cvtest

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.8889	1.0000	1.0000	0.9444	0.9444	0.8889	0.8889	1.0000	0.8889	0.9444
2	0.8889	1.0000	1.0000	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
3	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
4	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
5	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
6	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
7	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
8	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
9	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
10	0.8889	1.0000	0.9444	0.9444	0.9444	0.8889	0.8889	1.0000	0.9444	0.9444
11										

```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=10
ka=0.01;
a0=0.5;
```



cvtr

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.9597	0.9194	1.0000	0.9758	0.9758	0.9677	0.9758	0.9516	0.9677	0.9677
2	0.9597	0.9919	1.0000	0.9919	1.0000	0.9919	0.9758	0.9839	0.9919	0.9516
3	0.9839	0.9919	1.0000	1.0000	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000
4	1.0000	0.9919	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
6	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
7	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
8	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
9	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
10	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
11										

cvval

10×10 double

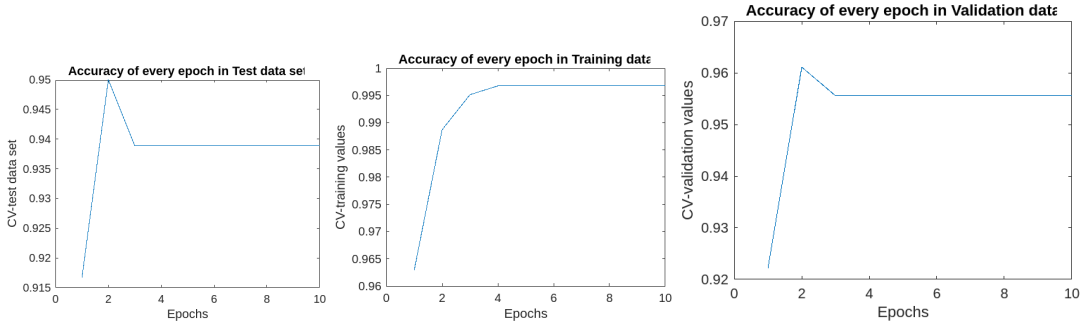
	1	2	3	4	5	6	7	8	9	10
1	1.0000	0.9722	0.9167	0.8611	0.9444	1.0000	0.9444	0.8889	0.9444	1.0000
2	1.0000	0.9722	0.9167	0.9722	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
3	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
4	1.0000	1.0000	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
5	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
6	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
7	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
8	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
9	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000
10	1.0000	0.9722	0.9167	0.9722	0.9444	0.9444	0.9444	0.9722	0.9722	1.0000

cvtest

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.9444	0.8889	1.0000	0.8889	0.9444	0.8889	0.7778	0.9444	1.0000	0.9444
2	0.9444	0.8889	1.0000	0.8889	0.8889	1.0000	0.9444	1.0000	1.0000	0.9444
3	0.9444	0.9444	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
4	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
5	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
6	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
7	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
8	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
9	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
10	0.9444	0.8889	1.0000	0.8889	0.8889	0.9444	0.9444	1.0000	1.0000	0.9444
11										

```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=10
ka=0.01;
a0=0.5;
```



cvtr

10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.9516	0.9919	0.9677	0.9758	0.9677	0.9274	0.9597	0.9758	0.9435	0.9677
2	0.9839	1.0000	0.9677	0.9919	1.0000	1.0000	0.9758	0.9919	0.9758	1.0000
3	0.9919	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9758	1.0000
4	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
5	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
6	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
7	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
8	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
9	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
10	1.0000	1.0000	1.0000	0.9919	1.0000	1.0000	1.0000	0.9919	0.9839	1.0000
11										

cvval

10×10 double

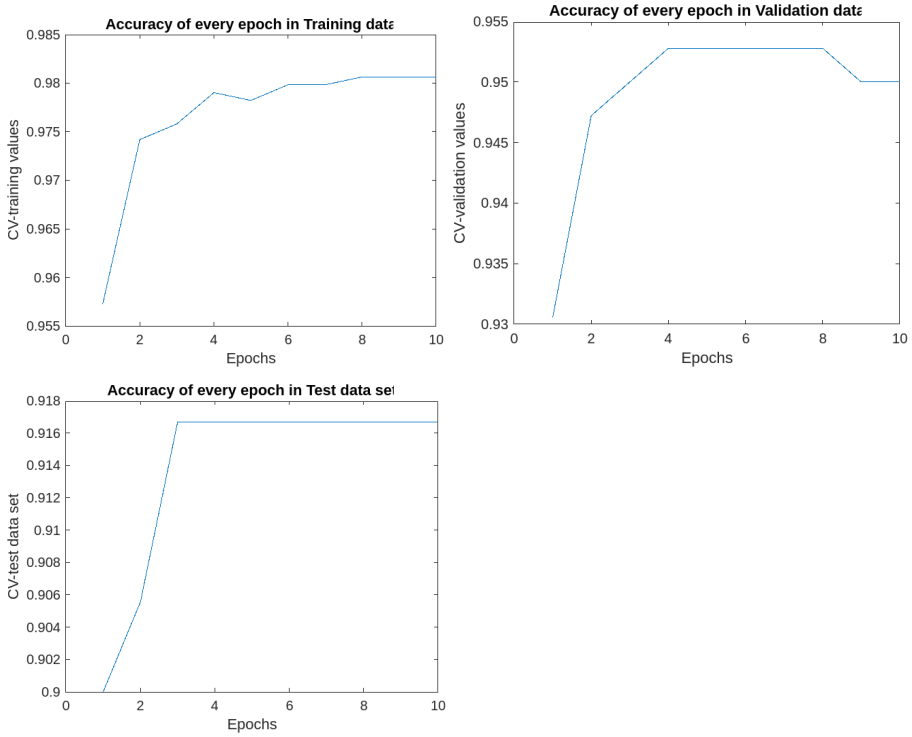
	1	2	3	4	5	6	7	8	9	10
1	0.9722	0.8889	0.9167	0.9722	0.8611	0.8056	0.9444	0.9444	0.9444	0.9722
2	0.9722	0.9167	0.9167	0.9444	0.9444	0.9722	1.0000	0.9722	0.9722	1.0000
3	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
4	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
5	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
6	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
7	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
8	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
9	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
10	1.0000	0.9167	0.8611	0.9444	0.9444	0.9722	0.9722	0.9722	0.9722	1.0000
11										

cvval cvtest

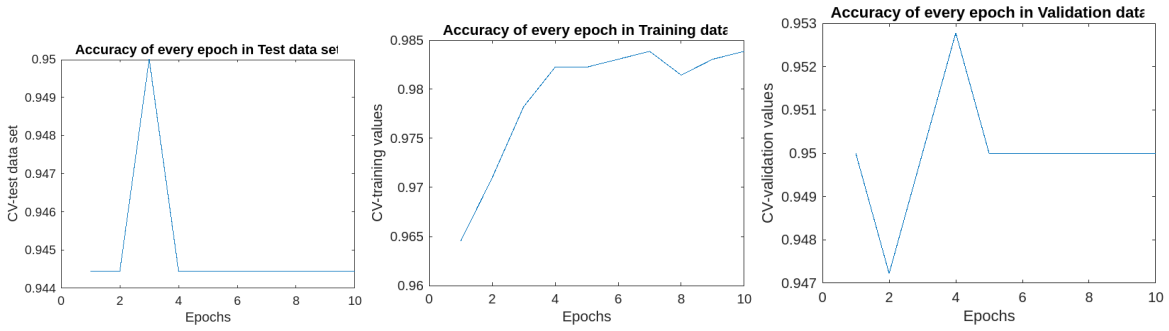
10×10 double

	1	2	3	4	5	6	7	8	9	10
1	0.7778	0.8889	0.9444	0.8889	1.0000	0.9444	0.8889	1.0000	0.9444	0.8889
2	0.9444	0.8889	0.8889	0.8889	0.9444	1.0000	1.0000	1.0000	0.9444	1.0000
3	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
4	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
5	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
6	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
7	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
8	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
9	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
10	0.8889	0.8889	0.8889	0.8889	0.9444	1.0000	0.9444	1.0000	0.9444	1.0000
11										

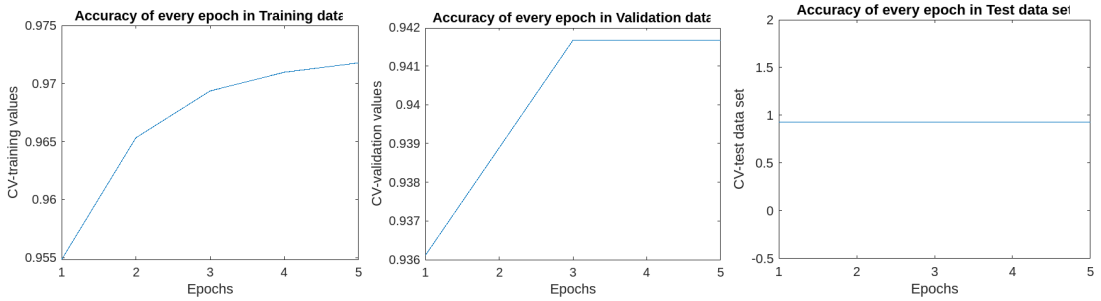
```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=10;
ka=0.1;
a0=0.5;
```



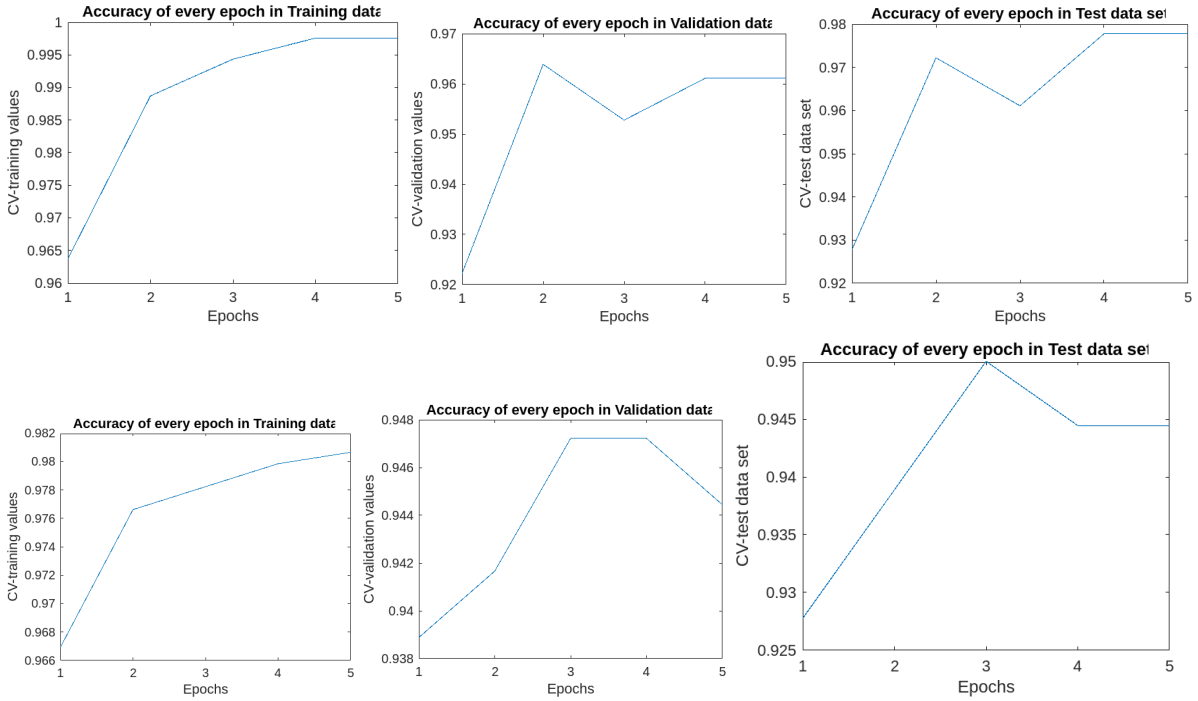
```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=10;
ka=0.1;
a0=0.5;
```



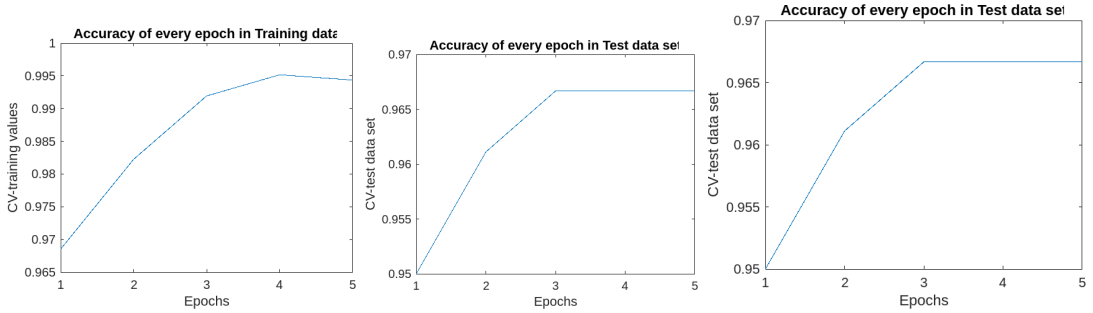
```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=5;
ka=0.1;
a0=0.5;
```



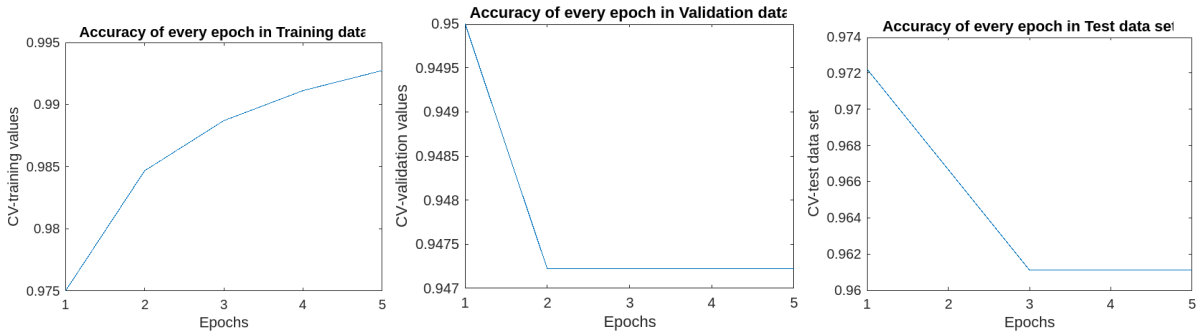
```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=5;
ka=0.01;
a0=0.5;
```



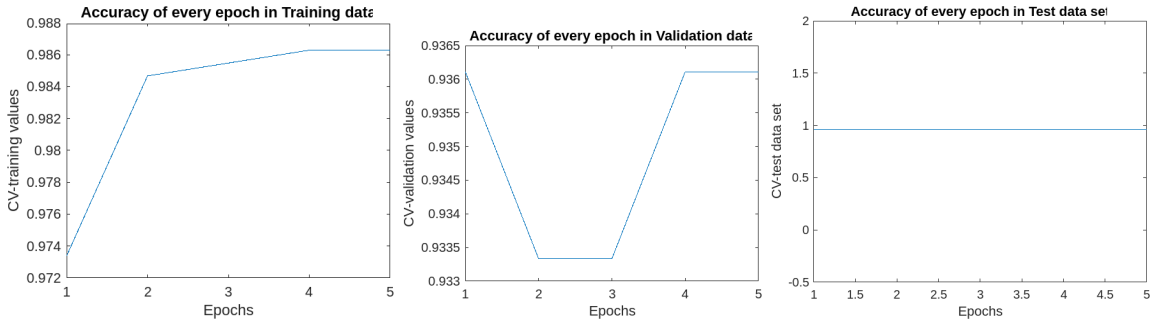
```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=5;
ka=0.01;
a0=0.5;
```



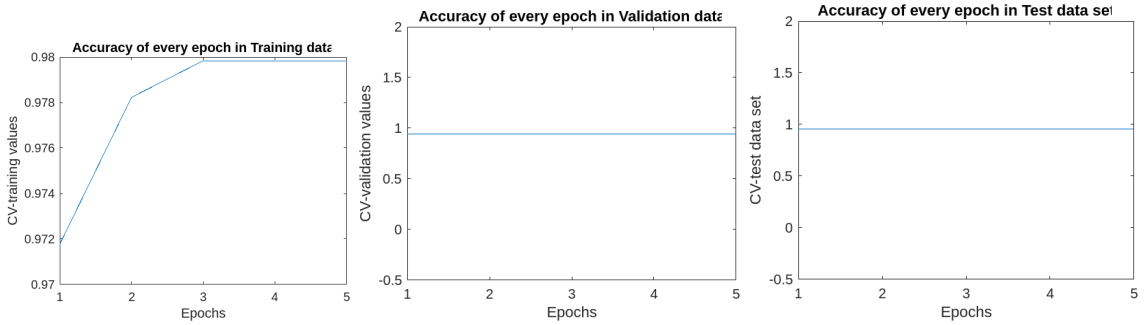
'WINE.TXT'
s=0.6;
ppc=2;
epochs=5;
ka=0.1;
a0=0.5;



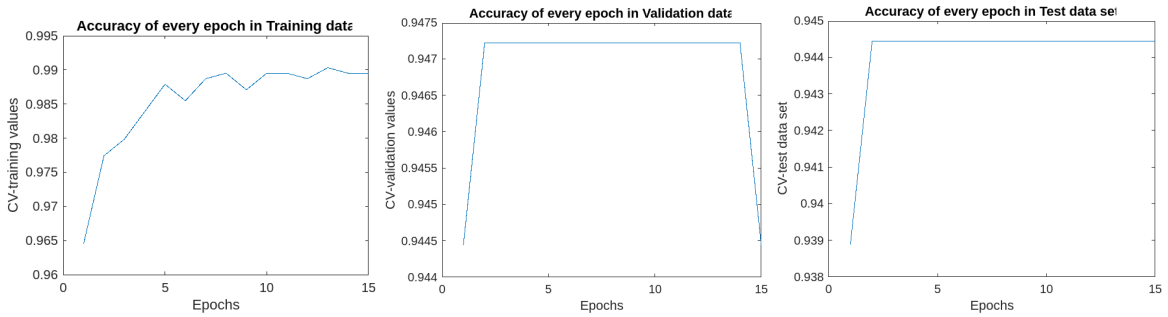
'WINE.TXT'
s=0.6;
ppc=5;
epochs=5;
ka=0.01;
a0=0.5;



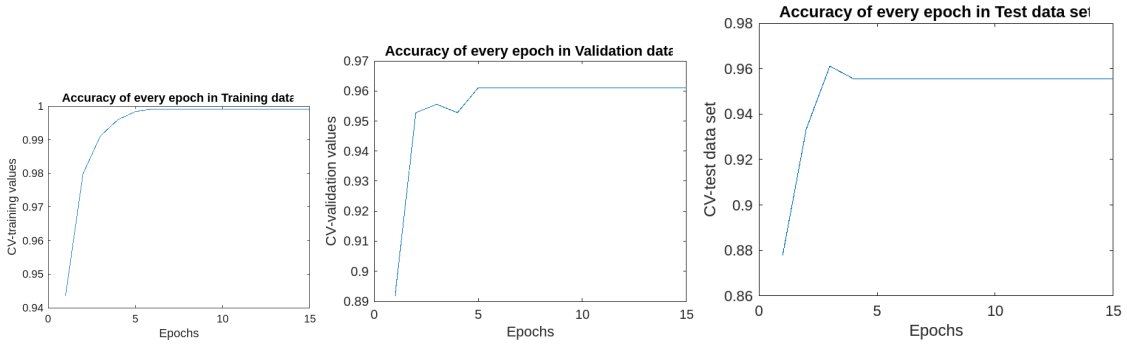
'WINE.TXT'
s=0.6;
ppc=5;
epochs=5;
ka=0.1;
a0=0.5;
Elapsed time is 2.580860 seconds.



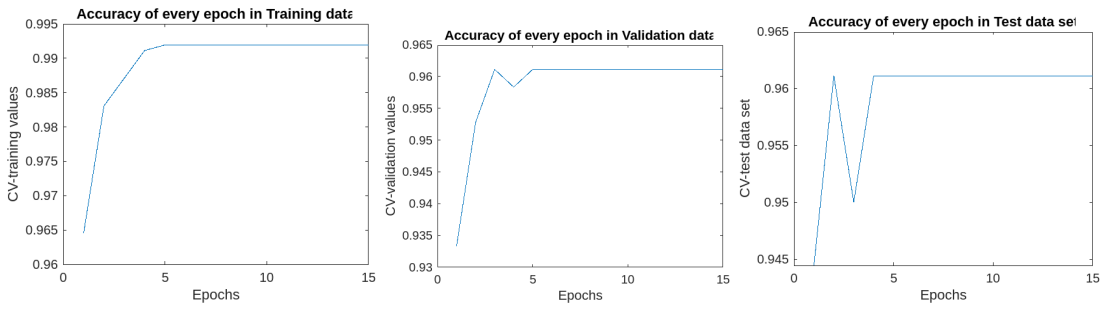
```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=15;
ka=0.1;
a0=0.5;
```



```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=15;
ka=0.01;
a0=0.5;
```

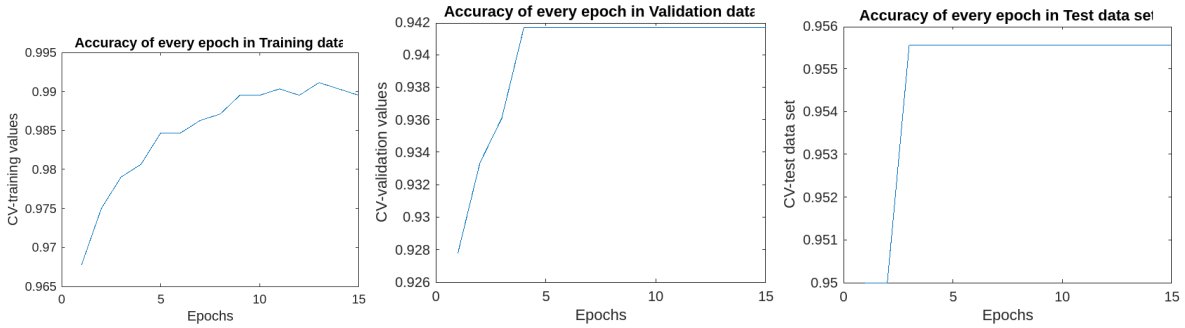


```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=5;
ka=0.1;
a0=0.5;
```

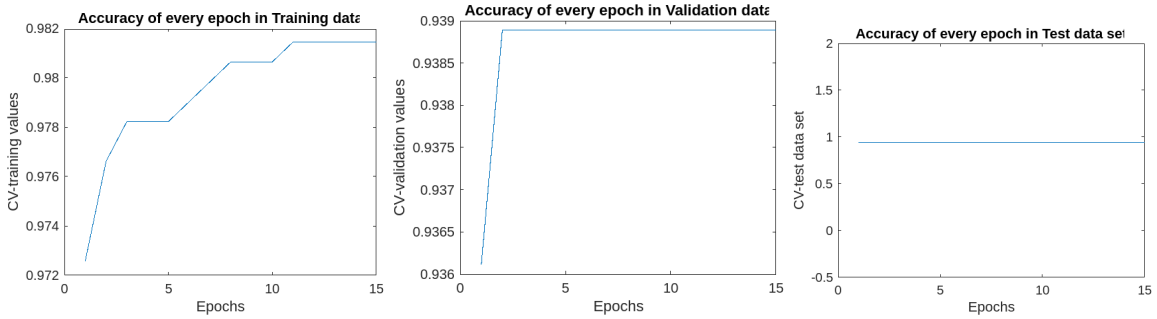


```
'WINE.TXT'
s=0.6;
ppc=2;
```

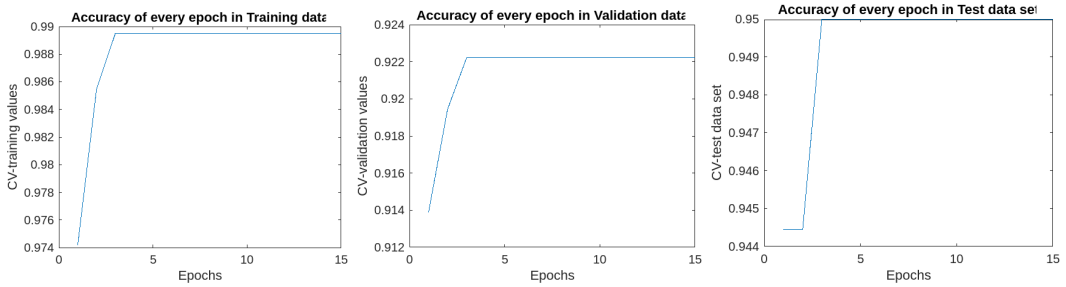
```
epochs=15;
ka=0.1;
a0=0.5;
```



```
'WINE.TXT'
s=0.6;
ppc=5;
epochs=15;
ka=0.1;
a0=0.5;
Elapsed time is 2.580860 seconds.
```



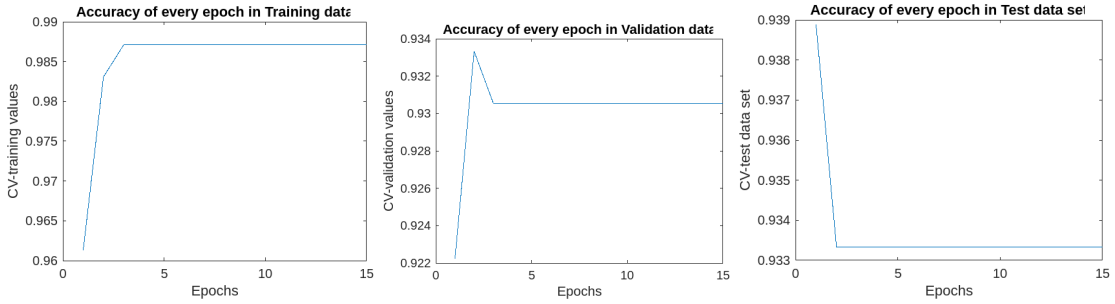
```
'WINE.TXT'
s=0.6;
ppc=5;
epochs=15;
ka=0.01;
a0=0.5;
```



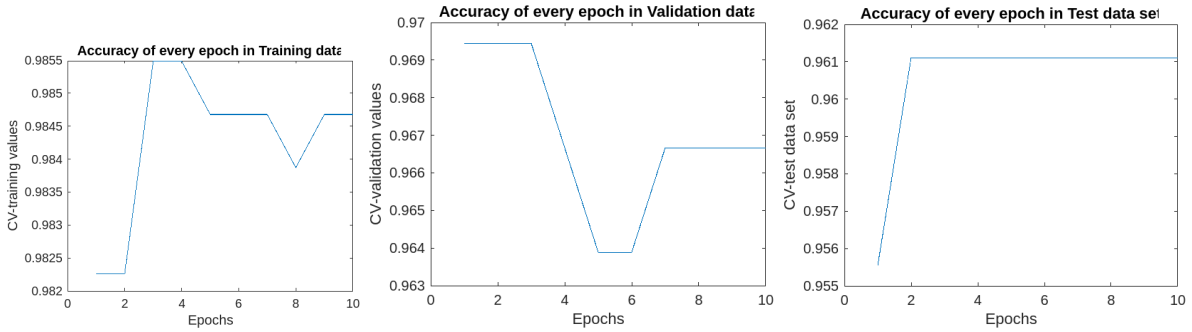

```
Elapsed time is 9.741575 seconds.
```

LFM

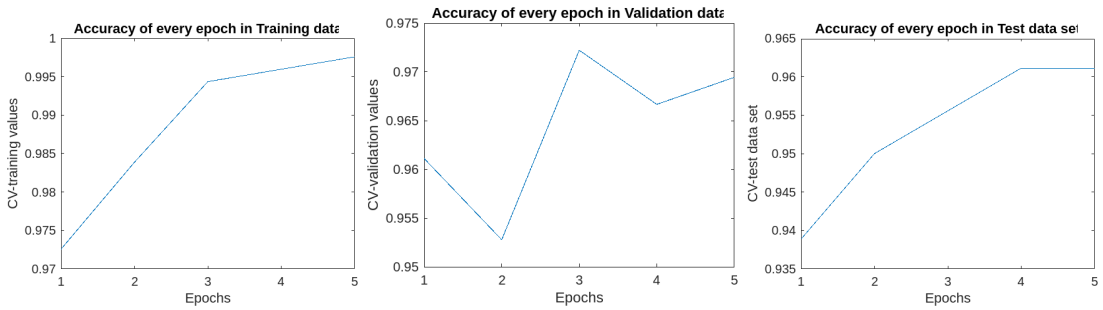
'WINE.TXT'
s=0.6;
ppc=5;
epochs=10;
ka=0.01;
a0=0.5;



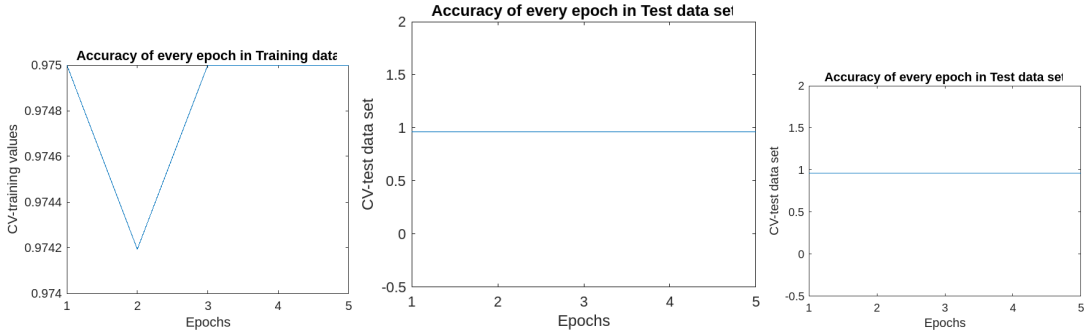
'WINE.TXT'
s=0.6;
ppc=5;
epochs=10;
ka=0.1;
a0=0.5;



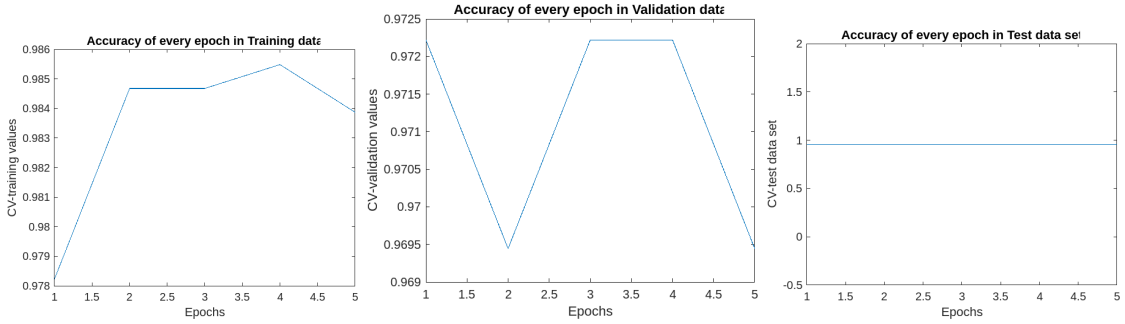
'WINE.TXT'
s=0.6;
ppc=1;
epochs=5;
ka=0.01;
a0=0.5;



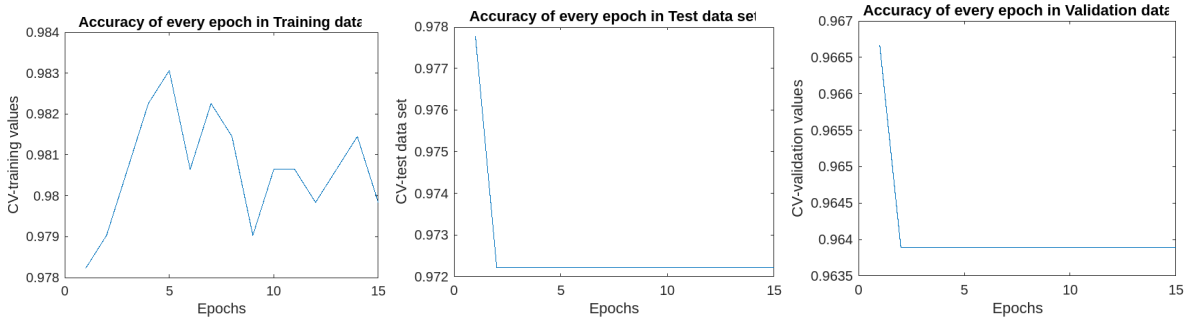
```
'WINE.TXT'
s=0.6;
ppc=1;
epochs=5;
ka=0.1;
a0=0.5;
```



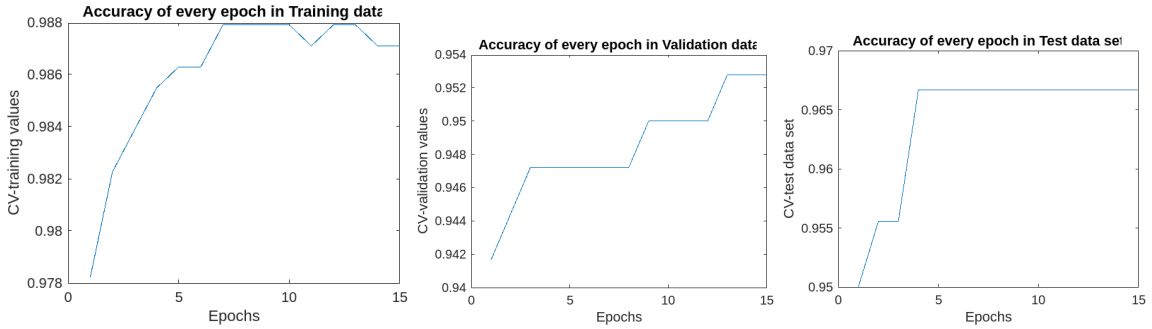
```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=5;
ka=0.1;
a0=0.5;
```



```
'WINE.TXT'
s=0.6;
ppc=2;
epochs=15;
ka=0.1;
a0=0.5;
```



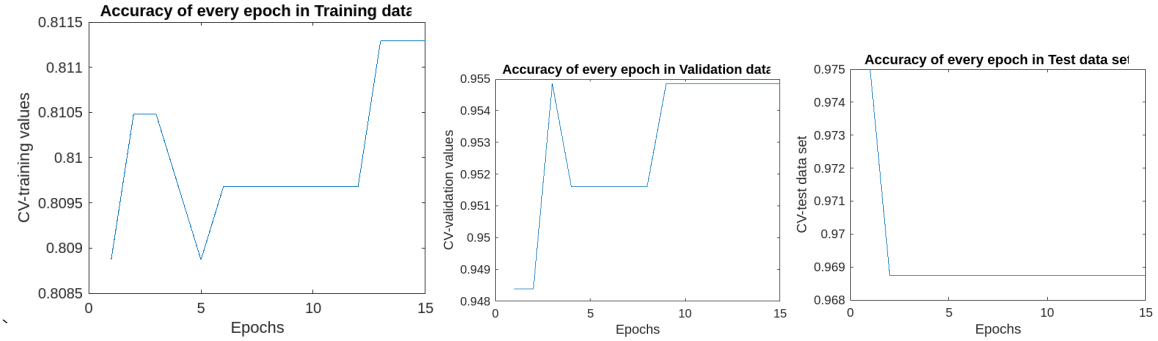
'WINE.TXT'
s=0.6;
ppc=5;
epochs=15;
ka=0.1;
a0=0.5;



IRIS

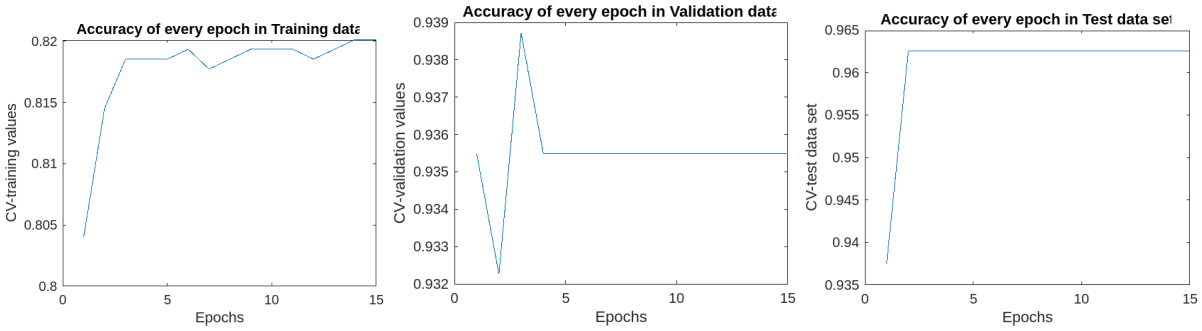
Ενδεικτικά για το iris data set παράχθηκαν τα παρακάτω αποτελέσματα για των αλγορίθμων lvq και lfm μετά από την εκπαίδευσή τους, με στόχο να παρατηρηθούν οι αλλαγές στο χρόνο εκπαίδευσης και αποτελεσματικότητας της διαδικασίας.

'IRIS.TXT' LVQ2
s=0.6;
ppc=5;
epochs=15;
ka=0.1;
a0=0.5;



Elapsed time is 9.708526 seconds.

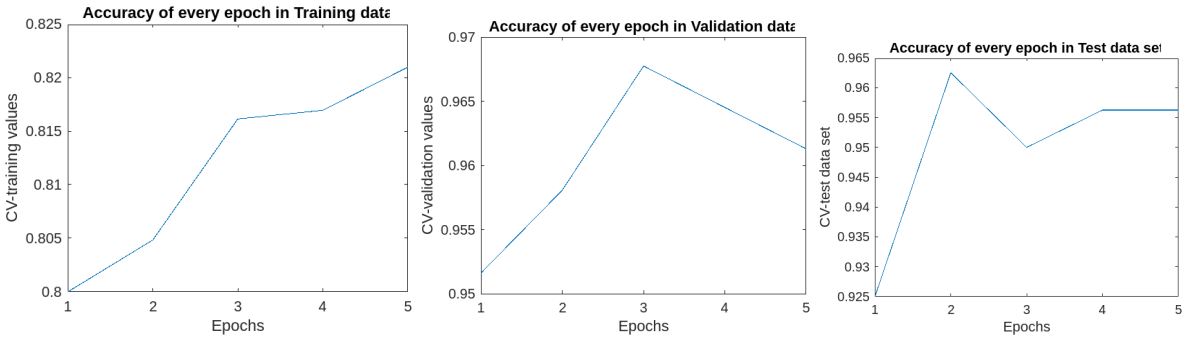
'IRIS.TXT' LVQ2
s=0.6;
ppc=5;
epochs=15;
ka=0.01;
a0=0.5;



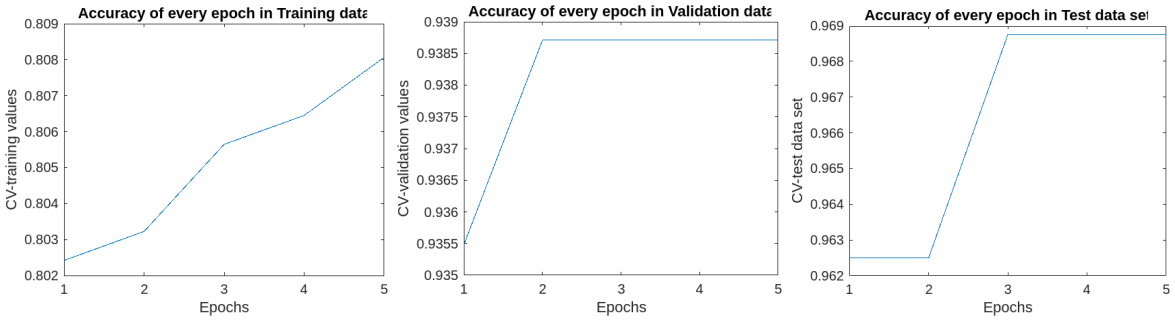
Elapsed time is 6.507313 seconds.

'IRIS.TXT' -LVQ2
s=0.6;
ppc=1;
epochs=5;
ka=0.01;
a0=0.5;

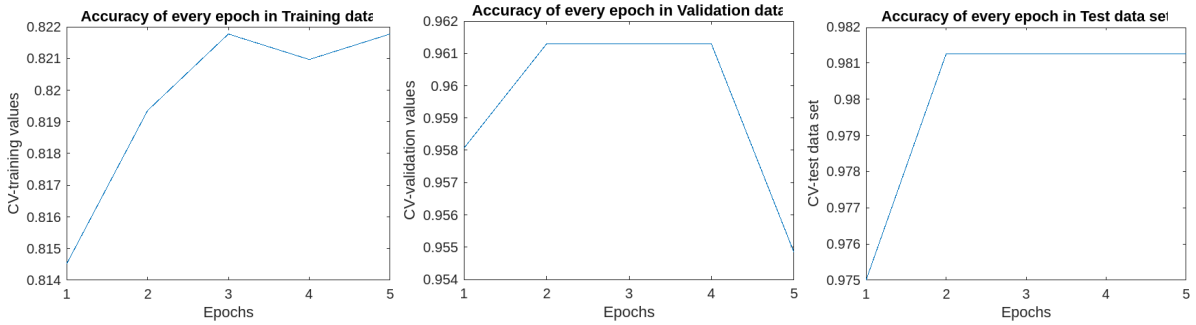
Elapsed time is 1.772000 seconds.



```
'IRIS.TXT' -LVQ2
s=0.6;
ppc=5;
epochs=5;
ka=0.01;
a0=0.5;
Elapsed time is 2.194269 seconds.
```

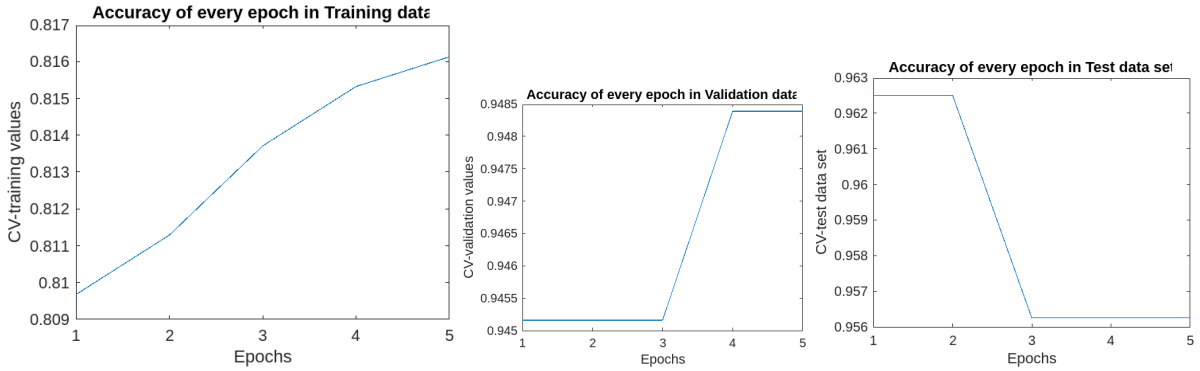


```
'IRIS.TXT' -LFM
s=0.6;
ppc=5;
epochs=5;
ka=0.01;
a0=0.5;
```



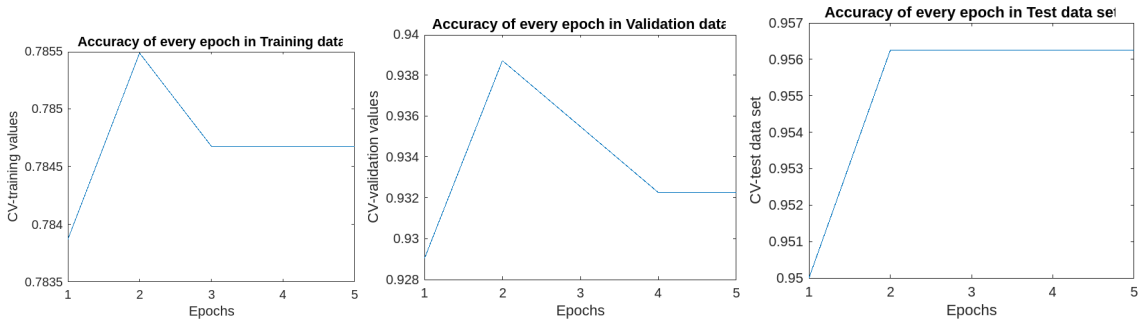

```
Elapsed time is 2.180718 seconds.
```

```
'IRIS.TXT' -LFM
s=0.6;
ppc=5;
epochs=5;
ka=0.1;
a0=0.5;
Elapsed time is 2.246311 seconds
```



```
'IRIS.TXT' -LFM
s=0.6;
ppc=2;
epochs=15;
ka=0.01;
a0=0.5;
```

Elapsed time is 1.769409 seconds.



'IRIS.TXT' -LFM
s=0.6;
ppc=;
epochs=5;
ka=0.1;
a0=0.5;
Elapsed time is 5.561552 seconds.

